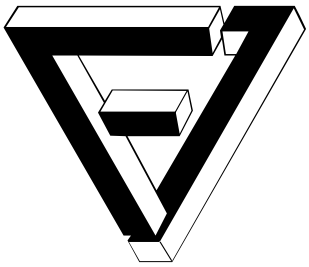
Masaryk University
Faculty of Informatics



Side-Effect Analysis Using a Static Single Use Program Representation

Master's Thesis

Samuel Malec

Brno, 2026

Declaration

Hereby I declare that this thesis is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, I used Claude Code (Anthropic, https://claude.ai) to write and review source code, to run and summarise experiments, and to help structure the text. After using this tool, I checked the content and accept full responsibility for it.

Supervisor: Petr Ročkai

Acknowledgements

Write your acknowledgements here.

Abstract

Write your abstract here.

Keywords

side-effect analysis, static single use, linear types, points-to analysis, ahead-of-time compilation, memory optimization, thread escape analysis, lock elision, GraalVM Native Image

Contents

1 Introduction 1

2 Background 2
- 2.1 Side Effects and Their Analysis . . . 2
- 2.2 Program Representations . . . 2
- 2.3 Graal and Its Memory Model . . . 2
- 2.4 Native Image . . . 2

3 Analysis of the Problem 3
- 3.1 What a Call Costs the Optimizer . . . 3
- 3.2 What Information Would Help . . . 3
- 3.3 Requirements . . . 3
- 3.4 Approach . . . 3

4 Explicit Effects 4
- 4.1 A Language with Heap Tokens . . . 4
- 4.2 Semantics . . . 4
- 4.3 Partitions . . . 4
- 4.4 Signatures . . . 4
- 4.5 What the Tokens Guarantee . . . 4
- 4.6 Exceptions and Deoptimization . . . 4
- 4.7 Concurrency . . . 4
- 4.8 From Signatures to Kill Sets . . . 4

5 Implementation 5
- 5.1 Architecture . . . 5
- 5.2 Exporting the Points-to Analysis . . . 5
- 5.3 Translation to the Token Form . . . 5
- 5.4 Signatures and Kill Sets . . . 5
- 5.5 Back into Graal . . . 5
- 5.6 Thread Escape in Native Image's Analysis . . . 5
- 5.7 Lock Elision and Ordinary Volatile Accesses . . . 5

6 Evaluation 6
6.1 Method . . . 6
6.2 What Graal Optimizes with the Facts . . . 7
6.3 Cost . . . 10
6.4 What Blocks the Analysis . . . 14
6.5 Soundness in Practice . . . 15
6.6 Discussion . . . 15
6.7 Threats to Validity . . . 17

7 Related Work 19
7.1 Side-Effect and Mod/Ref Analyses . . . 19
7.2 Memory in Compiler Representations . . . 19
7.3 Escape Analysis and Synchronization Removal . . . 19
7.4 Analyses for Ahead-of-Time Compilation . . . 19

8 Conclusion 20
8.1 Results . . . 20
8.2 Contribution . . . 20
8.3 Future Work . . . 20

Bibliography 21

A Electronic Attachments 22

B Proofs 23

C The Dialect of Graal IR in Cthulhu 24

D Reproducing the Results 25
D.1 The Tables from One Commit . . . 25
D.2 Other Measurements . . . 26

1 Introduction

2 Background

2.1 Side Effects and Their Analysis

2.2 Program Representations

2.3 Graal and Its Memory Model

2.4 Native Image

3 Analysis of the Problem

3.1 What a Call Costs the Optimizer

3.2 What Information Would Help

3.3 Requirements

3.4 Approach

4 Explicit Effects

4.1 A Language with Heap Tokens

4.2 Semantics

4.3 Partitions

4.4 Signatures

4.5 What the Tokens Guarantee

4.6 Exceptions and Deoptimization

4.7 Concurrency

4.8 From Signatures to Kill Sets

5 Implementation

5.1 Architecture

5.2 Exporting the Points-to Analysis

5.3 Translation to the Token Form

5.4 Signatures and Kill Sets

5.5 Back into Graal

5.6 Thread Escape in Native Image's Analysis

5.7 Lock Elision and Ordinary Volatile Accesses

6 Evaluation

This chapter measures what Graal gains from the facts of our analysis when it compiles a program with GraalVM Native Image, and what the facts cost. It counts the loads Graal leaves, times the programs, follows the control flow, and measures the build (Sections 6.2 and 6.3); then it asks why the gain is as small as it is (Section 6.4) and whether the images are correct (Section 6.5). The facts are about memory only: no model of concurrency is used, so every lock and `volatile` access is assumed to order threads. The commands that regenerate the tables are in Appendix D.

6.1 Method

An *image* is a native image built from one program by one builder: Native Image from our fork of Graal, extended with the feature of Chapter 5. The feature takes the facts of Native Image's points-to analysis out of the build, our tool turns them into kill sets and partitions, and Graal's own phases use them. A *kill set* is the set of locations a call may write; without one, Graal assumes that the call may write anything and reloads every field after it. A *configuration* says which facts reach Graal (Table 6.1), and each is compared with the baseline that differs from it in exactly the facts under test.

Native Image runs its points-to analysis in two ways. The *default analysis* names an object by its type; the *allocation-site analysis*, selected with the builder's option `AnalysisContextSensitivity=allocsens`, names it by the place that allocated it. Partitions by object need the finer names; kill sets do not, so we measure them on both. The allocation-site analysis also changes the code Graal produces with none of our facts: on Are We Fast Yet [1] (AWFY) at `-O2` the calls left fall from 1,407 to 1,110 and the loads rise from 943 to 954. We therefore compare images only on the same analysis, and report the price of the allocation-site analysis separately (Section 6.3).

The four small programs were written to contain calls that write little, so the answer is known: they show that the mechanism works, not what a typical program gains. AWFY is the realistic program.

Table 6.1: The configurations. `graal` hands nothing to Graal, so apart from counting it is Native Image as released. `ceiling` is unsound: an upper bound, not a result.

Configuration	Analysis	Facts given to Graal	Compared with
`graal`	default	none	(baseline)
`fields`	default	kill sets, partitions by field	`graal`
`ceiling`	default	kill sets, unsound upper bound	`graal`, `fields`
`graal`	allocation sites	none	(baseline)
`objects`	allocation sites	kill sets and object-field partitions	`graal` on allocation sites

6.2 What Graal Optimizes with the Facts

6.2.1 Loads Left

On the four small programs, kill sets remove 12.5–32.5 % of the loads, and 62 % (29 to 11) in the methods that lose loads at all; the controls keep theirs, as they must. On AWFY they remove 40 of 943 loads (4.2 %) at `-O2` and 54 of 2,565 (2.1 %) at `-O3`, and the same 40 on allocation sites. Partitions by allocation site add nothing on AWFY: `objects` leaves exactly the loads that `fields` leaves, because they split one of its 234 fields. They pay where the program was written for them: on `Aliasing` they remove three loads more (16 to 13).

6.2.2 Run Time

A load that is no longer reloaded saves little by itself; it pays where it enables something more. We therefore time the programs. Table 6.6 gives AWFY at `-O3` on the default analysis.

The six benchmarks not listed are Bounce, CD, Json, Towers, Mandelbrot, and NBody; Mandelbrot and NBody do not change. **Queens** is the one clear win: −27.5 % to −27.8 % in every build, far outside the noise. Its recursive method `placeQueen` no longer has each call kill four array-reference fields, so their loads leave its loop (Section 6.2.4). **The total** is −0.1 % to −1.4 % (mean −0.8 %), against 0.0 % to +1.1 % for the other builds of `graal`: about 1 %, mostly Queens. Sieve and Storage are faster in all four builds, and we did not investigate why. **Permute** is +7.2 % to +8.5 % slower in every build. Its code changes (Section 6.2.4), but in an earlier experiment that moved all methods to a 16, 64, or 128-byte alignment instead of 32, `fields` was 6.5–7.1 % *faster* than `graal`, so we read it as code placement and not as an effect of the facts. Richards, List, Havlak, and DeltaBlue move by several percent between builds of one configuration, and their ranges for `fields` overlap those of the other builds of `graal`; we make no claim about them.

Table 6.2: The programs.

Program	What it is	Used for
`LoadStoreElimination`	34 methods: a call that writes little between a store and a load; 12 of them are controls that must keep their loads	loads
`Signatures`	17 methods, one per rule for what a call's signature says it may touch	loads
`Aliasing`	13 methods: two references that may or may not meet, with 4 controls	loads
`Allocation`	4 methods: a call that allocates enough to collect, which tests the virtual machine state a call kills	loads
AWFY	14 benchmarks over a small collection library of their own, 459 methods, pinned at commit `74306fe`; a `main` constructs each benchmark and prints whether it verified	loads, run time, cost
javac	the JDK's compiler, compiling the 79 sources of AWFY	cost, loads, run time
PetClinic	Graal's Native Image bundle of Spring PetClinic (Tomcat, H2, HikariCP), served under load	cost, loads, run time
Barista	eleven server applications of the Barista suite, release 0.6.6: a plain hello world, hello worlds on seven frameworks, two more Micronaut applications, and Spring PetClinic	cost, loads
Native Image's tests	its own JUnit suite: 23 images with 289 tests	robustness

6.2.3 The Ceiling

To learn whether more precise facts would help, we built the `ceiling` image, which leaves out the kill of everything the facts cannot name: identity hash codes, class initialization, synchronization, native and unresolved calls. It is unsound by construction, but it prints what the JVM prints on AWFY, and it is the most that precise kill sets could give Graal.

The ceiling removes three times as many loads as `fields` (172 against 54) and is not clearly faster: Queens, the one benchmark that gains, gains the same with `fields`, and Permute and Richards differ by several percent between builds of one configuration anyway. Making the remaining opaque calls precise would not make AWFY faster.

6.2.4 Control Flow and Loops

A load that no longer depends on a call may leave a loop, and a branch that depended on it may fold. We count the blocks, control-flow edges, ifs, and loops of every method at the end of Graal's low tier, and for every load whether it is in a loop. The

Table 6.3: Conditions of the measurements.

Item	Setting
Machine	the faculty server pontos04: two AMD EPYC 7371, 64 threads, 1 TB of RAM, Debian 13; the only machine, without hardware counters (`perf_event_paranoid` is 3)
Builder	GraalVM 25.5-dev, built from our fork of Graal on a JDK 25 with JVMCI
Run	2 November 2026, one run of `reproduce.sh` with `THREAD_ESCAPE=off`; tool `df40ed9`, fork `12284dadc70` (control flow: `acb8880`, `8a70f22a6ac`)
AWFY builds	`-O2` as the analysis builds it, and `-O3` with trivial inlining as a user would; they are different compilations, so only the columns of one build compare
Correctness	every image must print exactly what the JVM prints; a build whose output differs is a failure, not a data point
Counts	field loads, stores, and calls left in the program's methods at the end of Graal's low tier; static counts, which loop unrolling multiplies, not run time
Run time	14 benchmarks, 20 iterations, five runs, the first iteration dropped, the median; four cores of one NUMA node, nothing else running; images take turns run by run
Builds timed	`graal` and `fields` four times each, timed together; the first `graal` binary timed twice (A/A) shows the noise
Larger programs	3 November 2026, the stages `javac` and `frameworks` with `THREAD_ESCAPE=off`; tool `f185b31`, fork `308e7320996`; three builds of `graal` and of `fields` of each, one at a time
Build time	wall-clock seconds of one build at a time on an empty machine, configurations taking turns, five rounds; median and range, stage times from the builder's log

images are built with source positions, so that the same load can be found in two images.

Kill sets barely change the control flow as a whole (6 edges of 29,000), but they change it a lot in three methods. In `Queens.placeQueen` the loop, which contains the recursive call, loses its six loads of the four array-reference fields and then disappears; the method is 2.4 times larger (85 to 207 blocks) and has more edges, because its ifs depend on data. In `Permute.benchmark` the loop disappears too, and most branches with it (49 to 5 ifs); we did not establish why. We read the loops as unrolled by Graal once the loads were gone, but we did not read its decision. Loop-invariant code motion in the strict sense is rare: of the 33 loads that left a loop, 5 appear outside it and 28 are gone.

Table 6.4: Loads left in the program's methods; the percentage is the change against `graal` on the same analysis.

	Default analysis		Allocation sites		
Program	`graal`	`fields`	`graal`	`fields`	`objects`
`LoadStoreElimination`	40	27 (−32.5 %)	43	30 (−30.2 %)	30 (−30.2 %)
`Signatures`	16	12 (−25.0 %)	16	12 (−25.0 %)	12 (−25.0 %)
`Aliasing`	16	14 (−12.5 %)	18	16 (−11.1 %)	13 (−27.8 %)
`Allocation`	9	7 (−22.2 %)	9	7 (−22.2 %)	7 (−22.2 %)
Four small programs	81	60 (−25.9 %)	86	65 (−24.4 %)	62 (−27.9 %)
AWFY, `-O2`	943	903 (−4.2 %)	954	914 (−4.2 %)	914 (−4.2 %)
AWFY, `-O3`	2,565	2,511 (−2.1 %)	–	–	–

Table 6.5: The default-analysis loads of the small programs, `graal` to `fields`, by kind of method. *Controls* are the methods named `...Control`: a call in them may write the field, or a reference may alias another, so they must keep their loads. *Unchanged* are the loads of the other methods that do not change (helpers, `main`, inlined monitor code, and in `Signatures` the three methods that cross a monitor, a native call, and a `volatile` access). *Reduced* are the loads of the methods that lose loads. The three add up to the totals of the default analysis in the table above.

Program	Controls	Unchanged	Reduced
`LoadStoreElimination`	16 → 14	10	14 → 3
`Signatures`	–	12	4 → 0
`Aliasing`	5 → 4	9	2 → 1
`Allocation`	–	0	9 → 7
Four small programs	21 → 18	31	29 → 11 (−62.1 %)

6.3 Cost

6.3.1 AWFY

Kill sets add 1.3 seconds (5 %) to the build of AWFY, 1.9 of them in the analysis stage, where the facts are exported and the tool runs. The compile stage does not move (0.3 seconds against a spread of 6.9–8.1), and neither does the memory. The allocation-site analysis costs more than everything we add: `graal` on allocation sites takes 30.4 seconds against 24.1 (+26 %) and its analysis stage 2.7 times as long, and the partitions then add 2.9 seconds (+10 %) and gave nothing on AWFY. The price of the finer analysis grows with the program: javac builds in 67 seconds on the default analysis and in 288 on allocation sites (24 October, three rounds, nothing of ours in either).

Table 6.6: Run time of AWFY at `-O3`, default analysis, in percent against the first `graal` build (negative is faster).

Benchmark	`graal`, ms	A/A	other `graal` builds	`fields`, four builds
Queens	23.3	+0.4	−1.8 to −1.5	**−27.8 to −27.5**
Sieve	84.9	−0.8	−5.3 to −4.3	−11.3 to −8.1
Storage	42.3	−0.4	−0.2 to +0.3	−4.8 to −2.7
Permute	27.1	+1.1	−1.1 to +0.4	+7.2 to +8.5
List	39.5	0.0	+1.3 to +2.3	+1.5 to +4.1
Richards	97.6	+0.6	+3.3 to +9.9	+1.7 to +11.5
DeltaBlue	14.4	−0.3	+0.2 to +0.9	−0.1 to +1.7
Havlak	65.7	+0.6	+1.5 to +2.5	−2.1 to +2.4
Other six	27–83	−0.1 to +0.3	−0.4 to +1.3	−0.5 to +1.2
Total	699.6	+0.1	0.0 to +1.1	−1.4 to −0.1

Table 6.7: The ceiling on AWFY at `-O3`, default analysis: the loads left and the run time in percent against `graal`, one build of each; the loads of the four builds timed together differ from these by at most one.

	Loads	In loops	Total	Queens	Permute	Richards
`graal`	2,565	937	–	–	–	–
`fields`	2,511 (−2.1 %)	904	+0.4	−27.9	+6.5	+13.7
`ceiling`	2,393 (−6.7 %)	859	−1.5	−26.7	−7.6	−0.7

6.3.2 Larger Programs

AWFY has 459 methods; the programs for which Native Image is built have many more. Table 6.12 gives the build of javac, of the PetClinic bundle, and of the eleven Barista applications on the default analysis, `graal` against `fields`, three builds of each, one at a time on an empty machine, the configurations taking turns.[1] In every program the ranges of the two configurations do not overlap, so three builds tell the increment from the spread.

The plain hello world has 3 loads in its own code. Kill sets add 7 to 34 % to the build: 2.2 seconds to the plain hello world, 6.8 to 15.0 seconds (+15 % to +22 %) to the eight framework applications of ordinary size, and 30 and 44 seconds (+26 % and +34 %) to the two PetClinics, the bundle of Graal and the one of Barista. In every program the percentage is larger than on AWFY (+5 %), and it grows with the program. The increment is in the analysis stage, where the facts are exported and the tool runs; the compile stage moves by −1.9 to +0.4 seconds. The analysis column can exceed the total: on PetClinic the stage grows by 33.8 seconds and the build by 30.0, because the other stages together are 3.8 seconds shorter in the `fields` builds.

[1] The Barista suite is at `https://github.com/barista-benchmarks/barista`. Its two Quarkus applications are left out: they do not build with Native Image from our fork even without our feature.

Table 6.8: Control flow of AWFY at `-O3`, default analysis, summed over the 185 methods the images compile.

	`graal`	`fields`
Blocks	21,197	21,201
Control-flow edges	28,997	29,003
Ifs	8,515	8,514
Loops	602	599
Methods with other control flow	–	9

Table 6.9: The methods whose control flow changes most, `graal` to `fields`. The other five of the nine change by at most 12 edges each.

Method	Blocks	Edges	Ifs	Loops
`Queens.placeQueen`	85 → 207	119 → 299	39 → 97	1 → 0
`Permute.benchmark`	106 → 15	152 → 19	49 → 5	1 → 0
`Permute.permute`	31 → 16	41 → 19	13 → 6	–
`Queens.queens`	120 → 126	169 → 176	–	2 → 1

That is within their spread between builds of one configuration (the compile stage of the three `graal` builds took 20.9–23.1 seconds, of the three `fields` builds 20.4–21.7). The peak memory of the servers differs by at most 0.3 GiB; javac's rises from 6.2 to 7.6 GiB, AWFY's by 0.1.

javac is the exception in how the increment divides. Its analysis stage grows by 7.9 seconds (+13 % of the build), but the total by 14.0: the rest is in the stage that lays out the methods, which took 10.4–10.7 seconds in every `graal` build and in one of the `fields` builds, and 16.0 and 16.4 seconds in the other two. Three builds do not tell whether `fields` slows that stage or some builds are slow; the build with the fast stage took 69 seconds against 61–64 for `graal`, which is 7 seconds more. The analysis increment of 7.9 seconds is the one to cite.

The loads that `fields` removes are few: 0.4 to 0.9 % of the loads in the servers' own methods, and 3.1 % in javac (7,846 to 7,531 of them in loops). The calls left do not change (javac 38,299 to 38,301, PetClinic 278,122 to 278,171). We did not establish why the servers gain so little.

Run time can be told for two of the programs. javac compiles the 79 sources of AWFY in 0.240–0.241 seconds in the three `graal` builds and in 0.239–0.243 in the three `fields` builds (ten runs of five compilations each); the first `graal` image timed again differs by −0.3 %, so there is no difference. PetClinic served 4,085–4,344 requests per second in the three `graal` builds and 4,175–4,384 in the three `fields` builds (16 connections, five runs of 30 seconds each after a warm-up of 15); the first `graal` image measured twice differs by +0.4 %, the runs of one image by about 1 %, but

Table 6.10: The loads that `fields` removes from AWFY at `-O3`, by where they were. A load that left a loop and appears outside it was hoisted, peeled, or unrolled: the counts cannot tell which. The numbers are changes in the count of loads, `graal` to `fields`.

Loads	Change
in loops	−33
of these, left the loop and appear outside it	5
of these, gone (forwarded, merged, or folded)	28
entered a loop	0
outside loops (the 5 included)	−21
all loads	−54

Table 6.11: Build of AWFY at `-O3`, median of five rounds in seconds (range of the total).

Configuration	Total	Analysis	Compile	Memory, GiB
`graal`	24.1 (23.1–24.8)	4.4	7.4	5.3
`fields`	25.4 (24.8–25.6)	6.3	7.7	5.4
added	+1.3 (+5 %)	+1.9 (+43 %)	+0.3 (+4 %)	+0.1
`graal` on allocation sites	30.4 (30.3–32.6)	11.8	7.4	5.5
`objects` on allocation sites	33.3 (32.6–34.1)	13.7	7.8	5.5
added	+2.9 (+10 %)	+1.9 (+16 %)	+0.4 (+5 %)	0.0

the builds of one configuration by 6.3 % (`graal`) and 5.0 % (`fields`). The medians differ by +2.2 %, which is within the spread between builds, so it is not a gain. No Barista application was timed: with at most 0.9 % fewer loads there is nothing to see through the spread between builds.

6.3.3 Allocation Sites

Partitions by object need Native Image's allocation-site analysis, which costs Native Image itself. Table 6.13 gives that price from the builds we have: earlier runs, each row from one run, with the code and the model of concurrency of its day, so the numbers do not compare with the other tables. The allocation sites multiply the build by 1.3 on AWFY and by 4.3 on javac. On PetClinic a build with allocation sites and a cutoff of 100, with nothing of ours in it, took 1,830 seconds against 120, 15 times as long. On the ten Barista applications that built, an image with allocation sites, the cutoff, and our export and lock elision took 3.0 to 10.7 times the `graal` build; where we timed our steps (three applications), they were 6 to 17 seconds of it, so the factor is the price of the allocation sites. Spring PetClinic did not finish: our export ran out of memory. The factor grows with the program, and it is far above what the facts themselves cost (7 to 34 %).

Table 6.12: Build of the larger programs, median of three builds in seconds (range of the totals), by the build of `graal`.

Program	`graal`	`fields`	Added	Analysis	Loads
Spring PetClinic	128.0 (127.0–130.0)	172.0 (170.0–173.0)	+44.0 (+34 %)	+42.6	−0.9 %
PetClinic	116.0 (112.0–119.0)	146.0 (146.0–151.0)	+30.0 (+26 %)	+33.8	−0.9 %
play-scala	67.0 (67.0–68.0)	82.0 (79.0–84.0)	+15.0 (+22 %)	+14.9	−0.8 %
dropwizard	63.0 (62.0–63.0)	73.0 (72.0–80.0)	+10.0 (+16 %)	+12.2	−0.9 %
javac	62.0 (61.0–64.0)	76.0 (69.0–78.0)	+14.0 (+23 %)	+7.9	−3.1 %
spring-hello	62.0 (60.0–64.0)	73.0 (71.0–74.0)	+11.0 (+18 %)	+11.4	−0.8 %
micronaut-shopcart	61.0 (60.0–62.0)	72.0 (71.0–75.0)	+11.0 (+18 %)	+10.4	−0.6 %
micronaut-similarity	56.5 (56.2–57.3)	66.0 (64.0–67.0)	+9.5 (+17 %)	+10.0	−0.7 %
ktor	55.8 (55.2–57.6)	65.0 (64.0–67.0)	+9.2 (+16 %)	+10.1	−0.4 %
micronaut-hello	55.5 (55.5–56.3)	66.0 (65.0–67.0)	+10.5 (+19 %)	+9.6	−0.7 %
helidon	46.0 (44.9–46.5)	52.8 (51.0–52.8)	+6.8 (+15 %)	+5.9	−0.8 %
vertx	37.3 (36.0–37.9)	41.1 (40.6–43.5)	+3.8 (+10 %)	+5.1	−0.8 %
plain hello world	29.9 (29.1–30.2)	32.1 (31.7–32.5)	+2.2 (+7 %)	+2.6	–

6.3.4 Where the Time Goes

The added time is our work in the analysis stage. Table 6.14 divides it into its steps: the facts are collected from Native Image's analysis (together with a text report of them, which these builds discard), the facts are written to the export file, and the tool computes the kill sets from it. The three add up to the increment of the analysis stage in every program, to within 0.8 seconds. Collecting the facts is the largest step in all programs but the two PetClinics, where writing the export is (14.9 of 33.6 seconds on PetClinic, 17.6 of 42.5 on Spring PetClinic); the tool takes 15 to 30 %. Applying the kill sets happens in compilation, on several threads, and adds at most 2.3 seconds of thread time. The last row is AWFY built with partitions by object on allocation sites, against `graal` on allocation sites: its steps are as large as those of `fields`, and the check that every graph keeps its partitions consistent adds 0.4 seconds of thread time. No other program was built with partitions by object in this run, so no other program has such a row.

6.4 What Blocks the Analysis

The gain on AWFY is small because most of its methods remain opaque: of the 459 methods of the program, 199 still kill everything. Table 6.15 shows what leaving out the kill of each reason would free.

No single reason is enough. Each of the 187 methods that remain after the first three reaches all of an unresolved call, a transition to native code, a raw write, and a snippet, and the reasons form a chain: a method that may build an exception, or call

Table 6.13: Build time of Native Image on the default analysis and with allocation sites, in seconds, and the ratio.

Program (builds)	Default	Allocation sites	Times
AWFY (five rounds)	24.1	30.4	1.3
javac (three rounds)	67	288	4.3
PetClinic (one build)	120	1,830	15
Spring PetClinic (one build)	138	out of memory	–
play-scala	72	771	10.7
dropwizard	69	537	7.8
micronaut-shopcart	66	647	9.8
spring-hello	65	567	8.7
ktor	63	364	5.8
micronaut-hello	62	550	8.9
micronaut-similarity	59	460	7.8
helidon	50	377	7.5
vertx	42	265	6.3
plain hello world	33	99	3.0

`equals` on an object of unknown type, reaches a lock, then class initialization, then a native call. The layers have to go together, and even then AWFY does not run faster (Table 6.7).

6.5 Soundness in Practice

Chapter 4 argues soundness for the model; the proofs are sketches in three places. The evidence of this chapter is of another kind, and Table 6.16 lists it.

This is evidence by controls and examples, not a proof.

6.6 Discussion

Benefit. Graal optimizes a little more with the facts, and in a specific way. Where it must assume that a call writes anything and the call sits in a loop, the kill set lets loads leave the loop and the loop change shape; Queens is that case, with −27.7 %. Elsewhere the gain is a handful of loads (2–4 % of AWFY's) that shows in the total as about 1 %. The small size is not a weakness of the implementation: the ceiling removes three times the loads and is no faster, and the opacity of AWFY has several layers, each of which would have to go.

Cost. The facts are cheap on AWFY: kill sets add 5 % to the build. On larger programs they add 7 to 34 % (Table 6.12), almost all of it in the analysis stage, and

Table 6.14: Our steps in the `fields` builds, and in the `objects` build of AWFY, in seconds (median).

	Steps					
Program	Facts	Write	Tool	Sum	Stage added	Apply
Spring PetClinic	12.3	17.6	12.7	42.5	+42.6	2.3
PetClinic	9.8	14.9	8.9	33.6	+33.8	1.7
play-scala	6.0	5.1	3.4	14.5	+14.9	1.1
dropwizard	6.0	3.2	2.3	11.5	+12.2	0.6
javac	4.8	1.4	1.2	7.4	+7.9	0.5
spring-hello	5.6	3.4	2.5	11.5	+11.4	0.7
micronaut-shopcart	4.7	3.7	2.6	11.0	+10.4	0.9
micronaut-similarity	4.2	2.9	2.1	9.3	+10.0	0.7
ktor	5.0	2.5	1.9	9.4	+10.1	0.6
micronaut-hello	4.2	2.9	2.1	9.2	+9.6	0.7
helidon	3.1	1.8	1.2	6.0	+5.9	0.3
vertx	3.3	1.0	0.9	5.2	+5.1	0.3
plain hello world	1.9	0.5	0.5	2.9	+2.6	0.1
AWFY	1.5	0.2	0.3	1.9	+1.9	0.1
AWFY on allocation sites, `objects`	1.5	0.2	0.3	2.0	+1.9	0.3

Table 6.15: Methods of AWFY (of 459) freed from killing everything when the kill of the named effects is left out of the kill sets (export of 22 October).

Left out	Methods freed
identity hash codes, synchronization, unresolved calls, or other effects, each alone	0
class initialization alone	12
identity hash codes, class initialization, and synchronization together	12
… and unresolved calls	12
… and every other effect	199

they remove 0.4 to 3.1 % of the loads. The allocation-site analysis that partitions by object need is dearer still (Table 6.13): it adds 26 % to the build of AWFY, multiplies that of javac by 4.3 and that of PetClinic by 15, and takes the Barista applications 3.0 to 10.7 times as long; the partitions gained nothing on AWFY. Kill sets on the default analysis are therefore the affordable combination.

Not shown. We did not time the Barista applications, and PetClinic's throughput differs less between the configurations than between builds of one, so the larger programs show a cost and no run-time gain. We did not count loads at run time.

Table 6.16: Evidence that the images are correct.

Check	Result
Output	every image of this chapter prints exactly what the JVM prints
Controls that must fail	a method claimed to write nothing gives 171 instead of 177 in `LoadStoreElimination` at `-O3`, so a false claim does not pass unseen
Native Image's own tests	with the feature and `-J-ea -J-esa`, its 23 JUnit images build and all 289 tests pass; the assertions found three defects that the examples did not: disabled flows asked for their objects, calls killed the initialization state, and a JSON escape failed on half a surrogate pair

6.7 Threats to Validity

One machine and few programs. Every timing is from one server with an AMD Zen 1 processor and without hardware counters. The realistic program for run time is AWFY, with javac and PetClinic as larger checks; the gains on the four small programs are by design.

Code placement. Native Image places every method at a multiple of 32 bytes, so a change in the size of one method moves all that follow it. Placement alone moves twelve of the fourteen AWFY benchmarks by 4–13 % with identical instructions, which is above the effect of the facts for all but Queens. We address it by timing four builds of each configuration together and by not citing the benchmarks that vary between builds.

Static counts. A load left is not a load executed, and loop unrolling multiplies counts; `Allocation`'s 66 stores are one store unrolled 64 times. That Graal unrolled the loops of `Queens.placeQueen` and `Permute.benchmark` is our reading of the counts.

One wrong baseline. In the first run of the control-flow measurement one `graal` baseline (AWFY on allocation sites at `-O3`) printed an incorrect result on every run of the binary. Four rebuilds of it and a complete second run, 25 images, were correct. That configuration hands nothing to Graal, so our facts are not the cause; we have not explained it. The tables use the second run.

Single builds and older numbers. The ceiling is one build of each configuration, three builds of each configuration are all that the larger programs have, the allocation-site builds of Table 6.13 are of 24 to 29 October (one build each for PetClinic and

Barista), and the export behind Section 6.4 is of 22 October. The `-O3` build on allocation sites was not run.

7 Related Work

7.1 Side-Effect and Mod/Ref Analyses

7.2 Memory in Compiler Representations

7.3 Escape Analysis and Synchronization Removal

7.4 Analyses for Ahead-of-Time Compilation

8 Conclusion

8.1 Results

8.2 Contribution

8.3 Future Work

Bibliography

[1] Stefan Marr, Benoit Daloze, and Hanspeter Mössenböck. "Cross-Language Compiler Benchmarking: Are We Fast Yet?" In: *Proceedings of the 12th Symposium on Dynamic Languages (DLS)*. 2016.

A Electronic Attachments

B Proofs

C The Dialect of Graal IR in Cthulhu

D Reproducing the Results

This appendix lists what is needed to regenerate the tables of Chapter 6. All commands are in the directories `scripts` and `graal-native` of the tool's repository; the builder is the Graal fork, which the repository contains as a submodule.

D.1 The Tables from One Commit

The script `reproduce.sh` produces the tables of Sections 6.2 and 6.3 in one run. It refuses to start unless the tool and the fork are exactly a commit (no changed tracked files, no untracked files) and the builder is not older than the fork's sources. With `ALLOW_DIRTY=1` it runs for testing and marks the results `NOT-REPRODUCIBLE`. The chapter measures no model of concurrency, which `THREAD_ESCAPE=off` switches off in the builder. The procedure of the run of 2 November, on a machine where nothing else runs, was:

```
# on the development machine, from clean, committed checkouts of the tool and of the fork:
scripts/server/sync-pontos.sh      # copies both trees to the server and stamps them with
    their commits

# on the server:
export JAVA_HOME=$HOME/.mx/jdks/labsjdk-ce-latest-jvmci-25.4-b23_amd64
export PATH=$JAVA_HOME/bin:$HOME/mx:$HOME/.local/bin:$PATH
cd ~/graal/substratevm && mx build # the builder must be newer than the sources
THREAD_ESCAPE=off ~/explicit-effects/scripts/evaluation/reproduce.sh          # loads, time
    , ceiling, cost
THREAD_ESCAPE=off ~/explicit-effects/scripts/evaluation/reproduce.sh control  # control
    flow and loops
THREAD_ESCAPE=off ~/explicit-effects/scripts/evaluation/reproduce.sh javac frameworks  #
    the larger programs
```

In `graal-native`, the results are written to a directory of `out/reproduction/`, named by the UTC time, the commit, and `no-thread-escape`: `provenance.txt` (which code), one file per stage, and `SUMMARY.txt`. Table D.1 lists the stages. The first four took 55 minutes on 2 November; `javac` and `frameworks` took 12 minutes and 2 hours 4 minutes on 3 November.

Table D.1: The stages of `reproduce.sh` without a model of concurrency. The first four run by default; the last three are named on the command line.

Stage	Section	What it runs
`loads`	6.2.1	the configurations on the four small programs and on AWFY at `-O2`, on both analyses, and the counts
`time`	6.2.2	`graal` and `fields` on AWFY at `-O3`, four builds each, timed together with the A/A control
`ceiling`	6.2.3	`graal`, `fields`, and `ceiling` on AWFY at `-O3`: loads left and run times
`cost`	6.3	every configuration of AWFY, five rounds, with the stage times and the peak memory
`control`	6.2.4	control flow, loops, and the place of every load, on the small programs and on AWFY at `-O3`
`javac`	6.3.2	`graal` and `fields` on javac, three builds each, the loads left, and the time javac takes to compile AWFY's sources with each image
`frameworks`	6.3.2	`graal` and `fields` on PetClinic and eleven Barista applications, three builds each, the loads left, and PetClinic under load with every image

D.2 Other Measurements

The results that `reproduce.sh` does not regenerate have their own scripts, listed in Table D.2. The loads of the run of 2 November equal those of two earlier runs of the same stages, of 29 and 31 October, to within one load.

Table D.2: Scripts for the results outside the stages.

Result	Script
Loads by kind of method	`scripts/evaluation/load_split.py` on the `counts.txt` of two images; `reproduce.sh loads` runs it
javac, price of allocation sites	`scripts/evaluation/cost-javac.sh`, three rounds
Native Image's JUnit tests	`mx native-unittest com.oracle.svm.test` with the feature and `-J-ea`
Placement of code	`programs/awfy/layouts.py` and `time.py`
Controls	`LIE=<method> ./run.sh LoadStoreElimination`; it must go wrong